

Android AAPT

Home

AAPT Command

ADB Commands

Home > Command

aapt dump xmltree

aapt dump xmltree chrome.apk res/layout/main.xml
Print the compiled xmls in the given assets. (res/layout/main.xml)

aapt dump xmltree chrome.apk res/xml/about_chrome_preferences.xml
(res/xml/about_chrome_preferences.xml)

```

E: merge (line=6)
E: org.chromium.chrome.browser.coordinator.CoordinatorLayoutForPointer (line=7)
A: id(0x010100d0)=@0x7f0a0104
A: layout_width(0x010100f4)=(type 0x10)0xffffffff
A: layout_height(0x010100f5)=(type 0x10)0xffffffff
A: layout_weight(0x01010181)=(type 0x4)0x3f800000
E: org.chromium.chrome.browser.compositor.CompositorViewHolder (line=13)
A: id(0x010100d0)=@0x7f0a00c2
A: layout_width(0x010100f4)=(type 0x10)0xffffffff
A: layout_height(0x010100f5)=(type 0x10)0xffffffff
E: FrameLayout (line=18)
A: id(0x010100d0)=@0x7f0a02aa

```

Figure 5: Generation of XML tree of APK file

```

Administrator: C:\Windows\system32\cmd.exe

C:\Users\kumargaurav\Desktop>dexdump.exe -l xml diva-beta\classes.dex | more
<api>
<package name="android.support.annotation"
>
<class name="AnimRes"
  extends="java.lang.Object"
  abstract="true"
  static="false"
  final="false"
  visibility="public"
>
<implements name="java.lang.annotation.Annotation">
</implements>
</class>
<class name="AnimatorRes"
  extends="java.lang.Object"
  abstract="true"
  static="false"
  final="false"
  visibility="public"
>
<implements name="java.lang.annotation.Annotation">
</implements>
</class>

```

Figure 6: Using DexDump to view code aspects of the Android app

of utilities and forensic tools to check apps regularly, and keeps users informed about suspicious apps. While it updates the apps whenever required, it also bans or blacklists suspicious apps, and notifies users about the same.

Two utilities that can be used to verify the different parameters and security aspects of Android apps, before they are installed, are briefly described below.

AAPT tool (<https://androidaapt.com/>)

This free and open source library is available to check an Android app before actual installation. It is also used for reverse engineering and fingerprinting of coding done at the backend of the Android app.

With a single instruction, the permissions of an APK file can be generated and then the user can see whether these are secure or not.

Figure 4 displays all the associated permissions of the APK. It is clear from the figure that many permissions are very sensitive, and a decision can be taken whether to install this app or not.

The complete XML tree of the Android code is also available with the *xmltree* keyword in the instruction. In this way, the backend code can be analysed and forensic evaluation of the app can be done.

DexDump utility (<https://github.com/benbenmushi/dexdumpWindows>)

In addition to the AAPT tool, the DexDump utility can be used to see the backend script and code of an Android app. Here, DEX refers to Dalvik Executable, which ensures the successful execution of the Android app. DEX is the executable file with the compiled code of the Android app.

With every Android app, the DEX file can be generated by unzipping (extracting) the APK file. Using the DexDump utility, the .dex file is scanned and then fingerprinting of the code is done. The DexDump utility is used to view different perspectives of the code to find out if there is any malware or suspicious instruction in the Android app. **END** 

References

- [1] Statista Research Report, <https://www.statista.com/topics/876/android>
- [2] Top 10 Most Notorious Hackers of All Time, Kaspersky, <https://www.kaspersky.com/resource-center/threats/top-ten-greatest-hackers>
- [3] Significant Cyber Incidents, Center for Strategic and International Studies (CSIS), <https://www.csis.org/programs/strategic-technologies-program/significant-cyber-incidents>

By: Dr Gaurav Kumar

The author is the MD of Magma Research and Consultancy and is associated with various academic institutes, where he delivers expert lectures and conducts technical workshops on the latest technologies and tools.